# SYNOPSIS

## 1. Project Title

**Legends of the Forgotten Realm** – A Command-Line Turn-Based RPG in Python

## 2. Introduction & Objective

The objective of this project is to design and develop a fully playable, text-based Role-Playing Game (RPG) entirely within the command-line interface (terminal) using Python 3.x. The project demonstrates mastery of core programming concepts such as modular programming, object-oriented design, loops, conditional statements, data structures (lists, dictionaries, tuples), file handling (json), and procedural randomness (random).

## 3. Storyline / Game Concept

The kingdom of Eldoria has fallen into darkness, overrun by monsters across diverse regional environments (Village, Forest, Cave, Mountain, Desert, Ruins, Castle, Volcano, Frozen Mountain, Sky Temple, and the Demon Realm). The player takes on the role of a customized hero aiming to progress through **100 levels**, defeat powerful regional bosses every 10 levels, and ultimately defeat the **Ancient Demon King** to restore peace to the realm.

## 4. Core Features & Architecture

- **Character Creation & Classes:** Players choose a name and one of four distinct classes—*Warrior, Mage, Archer, or Assassin*—each with specialized base stats (HP, Mana, Attack, Defense, and Critical Chance).
- **Weapon, Armor & Rarity Systems:** Features a 7-tier rarity hierarchy (Common to Legendary) spanning at least 20 unique weapons and 12 distinct armor sets that scale stats, drop rates, and value.
- **Potion System:** Includes at least 15 varied consumables across Health, Mana, Mixed, and Buff (temporary stat booster) categories.
- **Turn-Based Combat & Mechanics:** A structured combat loop featuring standard attacks, unique class skill trees (with mana costs), critical hit calculations, defense stances (50% damage reduction), and potion usage.
- **Progression, Enemy Scaling & Bosses:** Features 25+ enemy types whose stats scale with the player's level, alongside 10 locked regional bosses appearing every 10 levels that act as progression gates.
- **Inventory, Shop & Economy:** Fully managed player inventory supporting equipment management, item drops via inverse-rarity probabilities, and a village shop for buying/selling gear.
- **Data Persistence:** Complete game state saving and loading handled efficiently via JSON files.

## 5. Technical Requirements & Implementation

- **Language:** Python 3.x
- **Interface:** Command Line / Terminal
- **Key Modules:** random (for procedural generation, drops, and crits), json (for save/load states), and structured custom modules (main, player, combat, items, enemies, world).
- **Error Handling:** Robust input validation using try-except blocks to handle invalid menu choices and missing files cleanly.

**SANDRA BOSE**